

# Week 1- Fundamentals

Condensed Study Notes

Generated: 2026-05-25 00:17 UTC

## Week 1- Fundamentals

Welcome to the start of our journey! This week, we're laying the groundwork for understanding how computers work.

Think of a computer like a very organized office. There's a main desk where the important work happens, and filing cabinets where information is stored.

This section is all about the very basic building blocks that make computers tick. We'll be looking at the core ideas that everything else is built upon.

Your instructors for this course are Dr. F Adamu-Fika and Ms. OV Okpoko.

## Learning Objectives

Hey there! As we dive into computing, we're going to focus on understanding how computers solve problems. Think of it like learning how to bake a cake.

First off, we'll learn about **algorithms**. An algorithm is just a step-by-step set of instructions for solving a problem or completing a task. It's like a recipe for baking that cake – it tells you exactly what to do, in what order, to get the perfect result.

We'll also explore different ways to create these step-by-step instructions. Imagine you have a huge pile of laundry to fold. You could:

- **Divide and Conquer:** Take a small basket, fold it, then move to the next basket. You're breaking a big job into smaller, manageable parts.
- **Greedy Approach:** Maybe you always grab the biggest item first and fold it, then the next biggest. You're making the best choice at each moment, hoping it leads to a good overall result.

We'll also look at how algorithms can be grouped based on *what* they're trying to achieve. For example:

- **Sorting Algorithms:** Like arranging your books alphabetically on a shelf.
- **Searching Algorithms:** Like finding a specific page in a very large book.

Finally, we'll start thinking about how to tell if one set of instructions (an algorithm) is *better* than another. Is one recipe faster for baking the cake? Does one method of folding laundry take less time? This is about understanding how efficient our instructions are.

## Definition of an Algorithm

Imagine you're trying to bake a cake. You need a recipe, right? An algorithm is very much like a recipe, but for solving problems, especially with computers.

It's essentially a step-by-step procedure or a formula for solving a problem. Think of it as a set of instructions.

There are a few important things that make a set of instructions a true algorithm:

- **Input:** An algorithm can take in information to work with. This is like the ingredients for your cake. It can take zero inputs (meaning it doesn't need any extra information to start) or more.
- **Output:** After following the steps, an algorithm produces a result. This is like the finished cake! It must produce at least one output.
- **Effectiveness:** Every single step in the algorithm must be basic enough that it can actually be carried out. You can't have a step like 'instantly teleport the cake to the table' if you're just using a regular oven. Each step needs to be doable.
- **Definiteness:** The instructions must be super clear and leave no room for doubt. Each step should be precisely defined, so there's no confusion about what to do next. Like 'add 2 cups of flour', not just 'add some flour'.
- **Finiteness:** An algorithm must eventually stop. It can't go on forever! It needs to finish after a certain number of steps, just like a recipe has a clear end point.

Examples of problems that use algorithms include sorting a list of names, searching for a specific item, or finding the quickest route between two places.

## Importance and application of algorithms

We've learned that algorithms are step-by-step instructions for solving problems. Now, let's see why they are so important and where we use them!

Think about how you get from your home to a new friend's house. You probably use a GPS app on your phone. That app uses an algorithm to figure out the fastest or shortest route for you. This is a real-world example of **navigation** (finding your way).

When you shop online, algorithms are working behind the scenes. They help search for products you want and even suggest other things you might like, which is part of **e-commerce** (buying and selling online).

One of the biggest reasons algorithms are important is that they **enable efficient problem-solving**. This means they help us solve problems much faster and more effectively than we could on our own.

Algorithms aren't just for maps or online stores. They can be used in many different areas, like managing computer **networks** (how computers connect and share information), creating **AI** (artificial intelligence, which makes computers seem smart), and organizing **databases** (large collections of information).

In fact, algorithms are a **core of computer science and software development**. They are the fundamental building blocks for almost everything computer programs do.

Another crucial area is **cybersecurity**, which is about keeping information safe. Algorithms are used for things like **encryption** (scrambling information so only the right people can read it) and **hashing** (creating a unique fingerprint for data to check if it's been tampered with).

## Classification by Function or Problem Type

Just like you might have different tools for different jobs in your toolbox, algorithms can also be grouped by what kind of problem they are designed to solve.

Here are some common types of algorithms:

- **Searching Algorithms:** Imagine you have a huge library and you need to find a specific book. A searching algorithm is like a librarian who has a systematic way to find that one book among all the others. They help you locate specific pieces of information within a larger collection of data.
- **Sorting Algorithms:** Think about alphabetizing a list of names or arranging numbers from smallest to largest. Sorting algorithms are designed to arrange data into a specific order, making it easier to work with or understand.
- **String Algorithms:** These algorithms are like text editors that can understand and manipulate words and sentences. They are used to process and work with sequences of characters, like finding specific phrases within a document.
- **Graph Algorithms:** Imagine a map of interconnected cities. Graph algorithms are used to solve problems related to these kinds of networks, where things are connected to each other. They help figure out the best routes or connections.
- **Decision Problems:** Sometimes, the goal isn't to find a specific answer, but to determine if something is possible or true. A decision problem is like asking a yes/no question about a situation. For example, "Can this puzzle be solved with these pieces?"
- **Optimisation Algorithms:** Picture trying to pack a suitcase to fit as many clothes as possible without going over a weight limit. Optimisation algorithms are used to find the very best solution when there are rules or limitations you need to follow.
- **Combinatorial Algorithms:** These algorithms deal with figuring out all the possible ways to arrange or select items. Think about planning a trip where you want to visit several cities and want to know all the different orders you could visit them in.
- **Numerical Algorithms:** When you need to do calculations, especially complex ones, numerical algorithms are the tools you use. They are for performing mathematical operations like finding the exact value of a complex equation or estimating areas.

## Classification by Algorithmic Paradigm

So far, we've learned that algorithms are step-by-step guides to solve problems. Now, let's talk about different *philosophies* or *strategies* that computer scientists use to design these guides. These strategies are called **algorithmic paradigms**. Think of them like different approaches you might take to clean your room: you could tackle it all at once, do one small section at a time, or break it into even smaller tasks and do those first.

Here are some of the main algorithmic paradigms:

**Brute Force:** *Imagine you've lost your keys and you decide to check every single possible spot\* in your house, no matter how unlikely. That's brute force!* This paradigm involves exhaustively searching through all possible solutions to find the correct one. An example is checking every possible combination of items to solve the **subset-sum problem** (figuring out if a group of numbers can add up to a specific target).

- **Divide-and-Conquer:** This is like breaking a big chore, say painting a house, into smaller, manageable tasks. You divide the problem into smaller subproblems, solve each one independently, and then combine their results to get the final answer.
- Think of sorting a huge stack of papers. You could split the stack in half, sort each half, and then merge the two sorted halves together. Algorithms like **Merge Sort** and **Quick Sort** use this approach.
- It can also be used for problems like calculating **Fibonacci sequences** (where each number is the sum of the two preceding ones) or the **Knapsack problem** (figuring out the most valuable items to fit in a bag).
- **Decrease-and-Conquer:** This is like simplifying a problem by making it slightly smaller, solving the simpler version, and then using that solution to solve the original problem. It's like taking one step back to see the bigger picture.
- There are a few ways to make the problem smaller:
  - **Decrease by a Factor:** Imagine you're searching for a specific word in a dictionary. Instead of starting from the first page, you open it roughly in the middle. If your word comes before the middle word, you only need to search the first half. This is like **Binary Search**, where you repeatedly cut the search space in half.
  - **Decrease by a Constant:** Think about adding items to a shopping list one by one. You add an item, then another, and so on. This is similar to **Insertion Sort**, where you build up a sorted list by taking one element at a time and inserting it into its correct position.
  - **Decrease with Variable Size Decrease:** This is a bit more flexible. It's like solving a puzzle by figuring out one piece and then using that to help solve other pieces. The **Euclidean algorithm** for finding the Greatest Common Divisor (GCD) of two numbers is an example, where the problem size reduces in varying amounts.
- **Transform-and-Conquer:** This strategy involves changing the problem into a different, often simpler, form that is easier to solve. It's like translating a difficult instruction manual into a language you understand better.
- An example is **Huffman Coding**, which is used for data compression. It transforms the original data into a more efficient code.
- Another is **Problem Reduction**, where you might transform a problem about finding the shortest path in one type of network into finding it in another, simpler type.
- **Representation Change** is also part of this, like using **balanced search trees** (data structures that keep themselves organized) to make searching faster.
- **Greedy Approach:** This is like making the best immediate choice at every step, hoping it leads to the best overall outcome. Imagine you're trying to get the most money by picking coins. You'd pick the largest coin available at each step. This approach makes the locally optimal choice at each step to find an overall optimal solution.
- **Dijkstra's Algorithm** for finding the shortest paths in networks where distances are always positive is a good example.
- **Prim's Algorithm** for finding a **Minimum Spanning Tree** (a way to connect all points in a network with the least amount of 'wire') also uses this.
- **Backtracking:** This is like exploring a maze. You go down a path, and if you hit a dead end, you backtrack (go back) to the last junction and try a different path. This paradigm incrementally builds solutions and abandons a candidate as soon as it determines it cannot lead to a valid solution.

- This is very useful for problems like the **N-Queens problem** (placing queens on a chessboard so none attack each other) or solving **Sudoku puzzles**.

**Approximation Algorithms:** Sometimes, finding the exact best solution takes way too long, even for a super-fast computer. In these cases, we use approximation algorithms to find a solution that's really close\* to the best, but much faster to compute. These are particularly helpful for **optimization problems** (finding the best possible solution) where the exact solution is computationally infeasible (too hard to calculate).

- An example is finding an **approximation for NP-hard problems** (a class of problems known to be very difficult to solve exactly) like the **Traveling Salesman Problem** (finding the shortest route that visits a list of cities and returns to the start).

## Structure of Algorithms

Think about how you might build with LEGOs. You have a big set of instructions, but within those instructions, you might have steps that tell you to repeat a certain action many times, like clicking the same type of brick onto a base over and over. The 'structure' of an algorithm is about how these steps are organized, not what the algorithm actually *does*.

One way to organize steps is by using repetition. This is often done with something called a **loop**.

- **Loops:** Imagine you have to clap 10 times. Instead of writing "clap" 10 separate times, a loop is like saying "clap, and do this 9 more times." It's a way to repeat a set of instructions until a specific condition is met.

Sometimes, an algorithm can be designed to solve a smaller version of the same problem by calling itself. This is called **recursion**.

- **Recursion:** Think of a set of Russian nesting dolls. To open all of them, you open the biggest one, and inside you find a smaller doll, which is just a smaller version of the same problem. You repeat the process of opening the doll until you find the smallest one.

The raw content mentions examples like calculating a factorial using both recursion and loops. A factorial is a mathematical operation (like  $5!$  which means  $5 \cdot 4 \cdot 3 \cdot 2 \cdot 1$ ). Calculating it can be done by repeating a multiplication, or by using the idea of calling the factorial function on a smaller number until you reach the base case.

## Examples of Recursive structure

Remember how we talked about recursion, where an algorithm calls itself to solve smaller versions of a problem? Let's look at some real-world examples to see how this works.

Imagine you have a big pile of laundry to fold. Instead of trying to fold it all at once, you could decide to fold one shirt, and then tell yourself to "fold the rest of the laundry" (which is a smaller pile!). You keep doing this – folding one item and then recursively telling yourself to fold the remaining smaller pile – until there's no laundry left.

Another way to think about it is like Russian nesting dolls. To open the largest doll, you open it, and inside you find a slightly smaller doll. To open *that* doll, you perform the same action: open it, and find an even

smaller doll inside. You continue this process, opening each doll to reveal a smaller one, until you reach the tiniest doll that can't be opened further. This is like a recursive process reaching its end.

In computing, a classic example is calculating the factorial of a number. The factorial of a number (like 5!) means multiplying that number by all the positive whole numbers less than it (5 4 3 2 1). A recursive way to think about this is: the factorial of 5 is 5 multiplied by the factorial of 4. And the factorial of 4 is 4 multiplied by the factorial of 3, and so on, until you get to the factorial of 1, which is just 1. You then multiply all these results back up.

## Algorithm EVENSUM

Let's talk about a specific algorithm called EVENSUM. Imagine you want to add up all the even numbers from the beginning up to a certain point.

This algorithm, EVENSUM, is designed to do just that.

Here's what it needs to work:

- **Input:** It takes a number, let's call it 'n'. Think of 'n' as telling us how far we should go. The problem states 'n' is a positive integer, meaning it's a whole number greater than zero (like 1, 2, 3, and so on).

Here's what it gives us:

- **Output:** It calculates and gives you the sum of the 'n'th even natural number. The problem clarifies that the very first even natural number is considered 0. So, if n=1, it's asking for the sum of the 1st even number (which is 0). If n=2, it's asking for the sum of the 2nd even number (which is 2, since the first is 0, then 2, then 4...).

## START

We're continuing our look at the EVENSUM algorithm, which is designed to calculate the sum of the first 'n' even natural numbers (starting from 0).

Think of the algorithm like a shopping list that needs to be checked off. We have a running total, and we go through items one by one until we've checked everything on the list.

Here are the specific steps we're looking at:

- **while (i ≤ n):** This is like saying, "Keep doing these next steps as long as the current item number (i) is less than or equal to the total number of items we need (n)." It's a condition that controls how long a process repeats.
- **running\_total = running\_total + (2 \* i):** This is where we update our running total. We take the current total (running\_total) and add to it the value calculated from (2 \* i). This (2 \* i) represents the specific even number we're adding in this step.
- **return EVENSUM n - 1 + (2 \* n):** Once the while loop finishes (meaning we've gone through all the necessary items), this step gives us the final answer. It's saying the final sum is the sum of the previous calculations (EVENSUM n - 1) plus the last even number we needed to add (2 \* n).

## Examples of Iterative structure

We've seen how algorithms can repeat steps using something called **iteration** (repeating a process multiple times). Let's look at a couple of examples to really get a feel for it.

Imagine you're trying to find a specific book in a library by looking at each shelf one by one, from the first shelf to the last. This is like an iterative process where you check one item, then move to the next, and the next, until you find what you're looking for or run out of shelves.

### Algorithm 1: Finding a Specific Item

Let's say you have a list of items, and you want to know if a particular item is on that list. An algorithm to do this would:

- Start at the beginning of the list.
- Look at the first item.
- If it's the item you're looking for, you're done! You found it.
- If it's not the item you're looking for, you move to the next item on the list and repeat the check.
- You keep doing this until you either find the item or you reach the end of the list without finding it.

This step-by-step checking and moving to the next item is a classic example of iteration.

### Algorithm 2: Counting Up

Another common iterative task is simply counting. Think about counting from 1 up to 10. You:

- Start with 1.
- Say '1'.
- Move to the next number, 2.
- Say '2'.
- Continue this process, adding 1 each time and saying the new number, until you reach your target number (in this case, 10).

Each time you add 1 and say the number, you're performing an **iteration** (one pass through the repeating part of the process). The algorithm repeats the action of 'add 1 and say the number' until the condition 'reached 10' is met.

## Pros and Cons

We've talked about different ways algorithms can be structured, like using loops (iteration) or having an algorithm call itself (recursion).

Now let's look at the good and not-so-good points of these two main structures:

### Recursive Algorithms:

- **Think of it like:** Imagine you're telling a story, and at one point in the story, you say, "And then, as the story goes on, it tells another story..." It can be really elegant and easy to follow the main thread of the

story.

- **What this means for algorithms:** Recursive algorithms can often be easier for us humans to read and understand because they mirror how we might break down a problem naturally.
- **The catch:** This elegant way of telling a story might require you to remember a lot of the previous story parts to keep track. In computing, this means recursive algorithms can sometimes use up more computer memory (the short-term workspace for calculations) than other methods.

### Iterative Algorithms:

- **Think of it like:** Imagine you're meticulously building a LEGO set, following each step in the instruction booklet one by one, and you don't need to keep referring back to complicated previous steps once you've completed them. You just keep going.
- **What this means for algorithms:** Iterative algorithms, which use loops to repeat steps, are generally more efficient when it comes to using computer memory. They tend to be more 'space-efficient'.

## Efficiency and Complexity

Imagine you have two different recipes for baking a cake. Recipe A takes 2 hours and uses 5 cups of flour. Recipe B takes 1 hour and uses 3 cups of flour. Both recipes make a cake, but Recipe B is more efficient because it uses less time and fewer ingredients.

Algorithms, like recipes, can also be compared based on how quickly they work and how many resources they use. This is what we mean by 'efficiency' and 'complexity'.

**Efficiency** is all about how well an algorithm does its job. It asks: Does it solve the problem quickly? Does it use resources wisely?

**Complexity** is about understanding how much of a resource an algorithm needs. This is super important when dealing with lots of data. If an algorithm is complex in terms of its resource needs, it might take a really long time or use up a lot of memory, especially if you give it a huge amount of data to work with.

There are two main ways we measure an algorithm's complexity:

- **Time Complexity:** This measures how long an algorithm takes to run. We often think of this as the 'runtime'. It's like timing how long each cake recipe takes to bake.
- **Space Complexity:** This measures how much memory an algorithm needs to work. It's like figuring out how much counter space and how many bowls each cake recipe requires.

## Real-World Examples of Algorithms

Algorithms are everywhere, helping us in our daily lives and in complex systems.

- **Finding Your Way (Pathfinding):** Imagine you're using a GPS app to get somewhere. The app uses algorithms to figure out the best route for you, considering traffic and road closures. Dijkstra's Algorithm is a famous example used in GPS systems to find the shortest or quickest path between two points.
- **Organizing Information (Sorting):** Think about how a library organizes its books so you can easily find what you're looking for. Algorithms can sort huge amounts of data, like customer lists or product

catalogs, so they are in a specific order. Quick Sort is an efficient algorithm often used to speed up tasks like organizing data in databases.

- **Looking Things Up (Searching):** When you type something into a search engine like Google, it needs to quickly find the most relevant web pages. Algorithms help with this. Binary Search is a very fast algorithm used in search engines to find specific information within a large, sorted list of data.
- **Keeping Secrets Safe (Cybersecurity):** When you send information online, like your credit card details, you want it to be private. Encryption algorithms are used to scramble this information so that only the intended recipient can unscramble and read it, keeping your data secure during transmission.

## Online resources

To help you learn more about algorithms, there are several excellent online resources available. Think of these like helpful guidebooks or interactive maps for your algorithm journey!

Here are some recommended sites:

- **VisuAlgo:** This website is like an interactive playground for algorithms. It uses animations to show how algorithms work, especially helpful for understanding how algorithms repeat steps (iterative structures) or call themselves to solve smaller problems (recursive structures). It's great if you learn best by seeing things in action.
- **MIT OpenCourseWare – “Introduction to Algorithms”:** Imagine getting access to lectures from a top university for free! This resource offers lessons based on a well-known textbook. It's particularly good for understanding the basic ideas of algorithms, the different strategies for designing them (paradigms), and how to measure their performance (complexity).
- **GeeksforGeeks – Basics of Algorithms:** This site is like a comprehensive library of algorithm explanations. You'll find articles covering introductory topics like sorting and searching, along with how to implement recursive and iterative structures. It also provides example code and clear explanations for each concept.
- **Coursera – “Algorithms Specialization” by Stanford University:** This is like a structured course you can take online. The first week focuses on the fundamental ideas of algorithms and how to think like an algorithm designer, taught by a professor.
- **Khan Academy – “Algorithms”:** This resource is designed to be very beginner-friendly, like a patient tutor. It offers easy-to-understand tutorials on basic algorithm concepts, how they are grouped (classifications), and how to start analyzing how fast they run (introductory complexity analysis). It also has interactive exercises to help you practice and remember what you learn.